

Documentation CI/CD complète — MicroCRM / Orion

Ce que ce document est. La synthèse en un seul endroit de la chaîne CI/CD de MicroCRM : d'où elle part, ce qu'elle fait aujourd'hui, et ce qu'elle ne fait pas. Il assemble une quinzaine de documents du dépôt, listés en §8.

Ce qu'il n'est pas. Une plaquette. Chaque case sans réponse porte « non implémenté » ou « non mesuré » avec sa raison, jamais une valeur plausible. Les chiffres qui suivent ont été relus dans le dépôt ; ceux qui viennent d'une exécution disent laquelle et quand.

État des sources au 2026-09-24. La référence est la branche `develop`. Quelques documents de la copie de travail portent encore des chiffres antérieurs au 2026-09-22 ; les écarts sont recensés en §8.4.

1. Introduction

1.1 Présentation du document

MicroCRM est une application de démonstration — un CRM réduit à la création, à l'édition et à la consultation de personnes rattachées à des organisations — livrée à l'équipe Orion avec une chaîne d'intégration qui s'arrêtait à la compilation. Ce document décrit la chaîne CI/CD construite depuis : le workflow de branches, la stratégie de tests, l'infrastructure décrite en code, les contrôles de sécurité, et l'automatisation des releases et du retour arrière.

1.2 Technologies principales

Catégorie	Outil	Rôle
CI/CD	GitLab CI (+ GitHub Actions pour le miroir)	10 étapes, 37 jobs, 14 fichiers. GitHub porte le code et les Pull Requests, GitLab exécute
Qualité	SonarQube / SonarCloud, SpotBugs + Find-Sec-Bugs, JaCoCo, PIT	Dette et Quality Gate, bugs de bytecode, couverture de lignes, force des assertions
Conteneurs	Docker multi-stage, Kubernetes, Kustomize, Helm	Une image par application, déploiement par overlays ; Helm est rendu et comparé, jamais appliqué
Cloud	aucun fournisseur — minikube local	Décision assumée, argumentée en §5.1 et transposée fournisseur par fournisseur en <code>ARCHITECTURE.md §10</code>
IaC	Terraform (provider <code>hashicorp/kubernetes</code>), Ansible	Terraform possède les namespaces, quotas, limites et politiques ; Ansible possède le poste et le cluster
Sécurité	OWASP Dependency-Check, Trivy	CVE des dépendances Java ; CVE d'image, secrets et misconfigurations
Performance	k6	Trois scénarios contre l'image qui vient d'être construite
Supervision	Elasticsearch, Filebeat, Kibana	Logs centralisés et tableaux de bord versionnés ; pas de métriques, pas d>alerting applicatif
Discipline de dépôt	husky, lint-staged, commitlint, Prettier, Spotless	Contrôles locaux avant le push

Coût de licence : nul. Tous ces outils sont gratuits dans l'usage qui en est fait ici ; l'investissement est en temps de mise en place et en montée en compétence (`VEILLE.md §7`).

2. Contexte et objectifs du projet

2.1 Contexte

L'entreprise et les équipes. Orion développe MicroCRM, une application full-stack — back Java / Spring Boot exposant une API REST, front Angular. Deux équipes s'en partagent le cycle de vie, et les deux ont répondu au sondage « pratiques et technologies » :

Équipe	Membres	Terrain
Dev	Roubina (lead), Sylvain (senior), Temim (junior), Josefina (stagiaire)	Angular/TypeScript, Spring Boot/Java, Gradle, NPM, JUnit, Karma, HyperSQL
Ops	Nico (lead), Maïa (senior)	Bash, Ansible, Apt/Debian, TestInfra, BashUnit, Docker, PostgreSQL

Le niveau déclaré n'est pas uniforme, et c'est structurant. L'équipe Dev se dit « bonne » en TypeScript/Angular, NPM, Karma et Docker, mais **débutante en Java/Spring Boot, Gradle, JUnit et HyperSQL** — c'est-à-dire sur toute la partie serveur. L'équipe Ops est « très bonne » en Bash, « bonne » partout ailleurs, et « moyenne » en PostgreSQL. **Aucune des deux équipes ne déclare de compétence Kubernetes**, alors que c'est la cible retenue : c'est une absence, pas une lacune mineure, et elle est traitée comme telle (§2.2, irritant I8).

Le cycle décrit par les équipes. Côté Dev, sept étapes du backlog à la génération des images. Côté Ops, trois : réception d'un numéro de version, analyse Trivy de l'image, déploiement manuel en commandes Docker sur l'environnement de démonstration. Entre les deux, **un email**.

Les points de douleur, tels qu'ils sont formulés.

Source	Ce qui est dit
Sondage Ops, Q3	« La majorité des opérations sont aujourd'hui manuelles et mériteraient d'être automatisées, notamment les contrôles de sécurité ainsi que le déploiement »
Sondage Dev, Q2	« Nous avons reçu dès le début un retour sur des CVEs présentes dans l'image, ce qui a retardé le premier déploiement » — le fameux <i>retour à l'expéditeur</i>
Sondage Dev, Q3	Projet au second sprint, opérations encore majoritairement manuelles
Sondage Ops, Q4	Besoin d'un dépôt d'images interne , pour ne plus dépendre de DockerHub
Sondage Ops, Q4	Besoin d'une analyse de sécurité des images en amont , avant transmission à l'équipe Ops
Sondage Dev, Q4	Besoin d' analyse statique et d'aide à la conception , précisément parce que l'équipe est débutante en Java
Sondage Dev, Q3bis	Trois artefacts produits , dont une image « tout en un » embarquant front et back

Ce que l'audit du dépôt ajoute (AUDIT.md §1-2). Le `.gitlab-ci.yml` livré tenait en **2 stages et 4 jobs** : `test-front`, `test-back`, `build-front`, `build-back`. C'est de l'intégration au sens strict — ça compile et ça teste, et ça s'arrête là. Étaient absents : toute analyse statique, toute mesure de couverture, tout contrôle de sécurité, **tout Dockerfile**, tout registry, tout déploiement, tout rollback, tout versionnage, tout hook local, toute règle de branche. Le `README.md` documentait des commandes `docker build` alors qu'aucun Dockerfile n'existait — une documentation fausse, c'est-à-dire pire qu'absente.

Les six frictions retenues (`AUDIT.md` §4) :

#	Friction	Effet
1	Aucun retour qualité avant la revue humaine	Le temps de revue part dans le style et les bugs simples
2	Détection tardive : tout remonte en CI, rien en local	Boucle de retour longue, minutes de calcul gaspillées
3	Mise en production manuelle	Non reproductible, dépendante d'une personne
4	Pas de rollback outillé	Temps de rétablissement non maîtrisé
5	Pas d'environnement de validation	Les régressions sont découvertes par les utilisateurs
6	Documentation absente ou fausse	Onboarding lent, dépendance aux personnes

Le goulot principal est l'absence totale d'automatisation après le `build` : tout le travail de vérification en amont n'est pas capitalisé, puisque la livraison repose ensuite sur des gestes manuels.

2.2 Objectifs

Cinq objectifs, chacun avec sa mesure et son état au 2026-09-24. Les valeurs « mesuré » viennent de `scripts/ci/collect_dora.py` (exécution du 2026-09-23, fenêtre de 30 jours, 50 pipelines) ou d'une exécution des suites du dépôt.

#	Objectif	Comment il se mesure	Cible	Mesuré aujourd'hui
01	Faire aboutir la chaîne de déploiement depuis la CI, sans geste manuel autre que la décision	<code>deployment_frequency</code> de <code>collect_dora.py</code>	> 0 / jour	0,1667 / jour — 5 déploiements réussis, atteint depuis le 2026-09-22
02	Ramener le délai entre la découverte d'une CVE et son retour à l'auteur de plusieurs jours à quelques minutes	Durée du job <code>package-* / dependency-check-back</code>	minutes	atteint — les 4 portes sont bloquantes depuis le 2026-09-19
03	Rendre la qualité du back pilotable par un seuil opposable	<code>coverage-gate</code> (JaCoCo) et <code>mutation-back</code> (PIT)	≥ 90 % lignes, ≥ 80 % mutants	97 % lignes, 100 % branches, 96 % mutants
04	Maîtriser le temps de rétablissement : le rollback doit être une commande, pas une improvisation	<code>time_to_restore_service</code> , et un rollback réellement joué	mesurable	2,14 h de médiane sur 2 observations ; un rollback de production joué le 2026-09-23
05	Faire descendre le taux d'échec des changements sous 50 %	<code>change_failure_rate</code> de <code>collect_dora.py</code>	< 50 %	66,67 % sur 9 tentatives — <i>non atteint</i>

⚠ **Ces chiffres n'ont aucune valeur statistique et il faut le dire avant de les commenter.** Neuf tentatives, deux observations de rétablissement, cinq réussites concentrées sur deux journées consécutives — celles où la chaîne a été débloquée. Ce sont des faits, pas des tendances, et la ligne de base existe parce qu'il en faut une, pas parce qu'elle serait stable (`MONITORING.md` §9.1, `docs/plan-optimisation-release.md` §5).

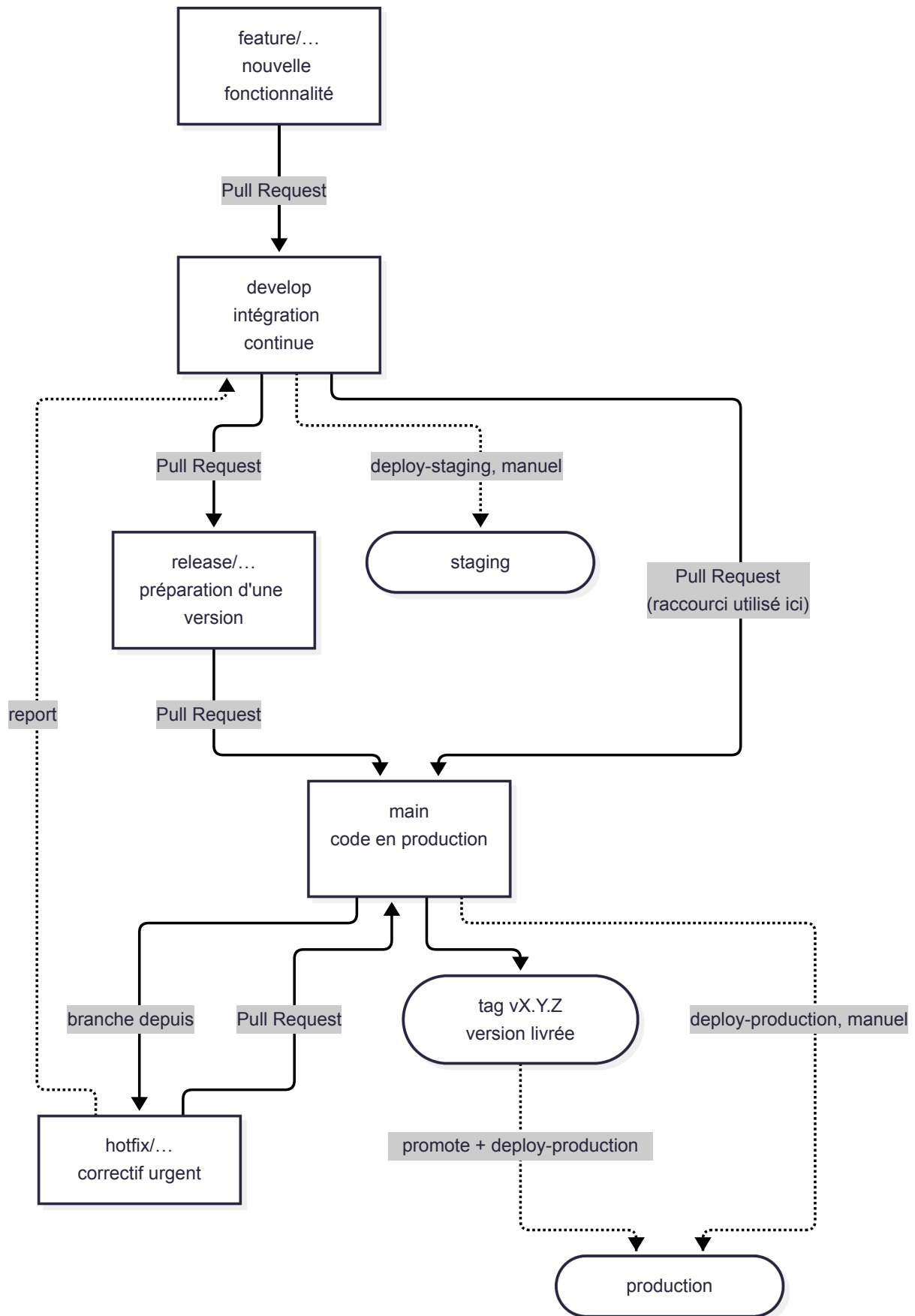
Le chemin détaillé — cinq vagues, un porteur et une preuve d'atteinte par action — est dans `docs/plan-optimisation-release.md` §4.

3. Workflow de branches

3.1 Modèle de branching choisi

GitFlow, avec `main` et `develop` permanentes et des branches de travail éphémères. Le modèle a été retenu parce qu'il donne une branche d'intégration distincte de la branche de production — condition nécessaire pour avoir un environnement de staging qui reçoive autre chose que ce qui part en production — et parce que `hotfix/` offre un chemin explicite pour corriger la production sans emporter ce qui traîne sur `develop`. Le coût assumé est sa lourdeur : sur un dépôt à un seul contributeur, la branche `release/` n'a jamais servi (§3.2).

3.2 Structure des branches



Ce que chaque branche déclenche (ARCHITECTURE.md §7, règles réelles dans .gitlab/ci/templates.yml) :

Branche	Rôle	Jobs déclenchés
main	Code en production	Tous, jusqu'au déploiement production
develop	Intégration continue	Tous, jusqu'au staging
feature/...	Nouvelle fonctionnalité	lint, test, quality, security — filtrés par périmètre
release/...	Préparation d'une version	Tous
hotfix/...	Correctif urgent en production	Tous
Tag vX.Y.Z	Version livrée	Tous sauf package-* : un tag promet, il ne reconstruit pas

L'écart entre le modèle et la pratique, parce qu'il se lit dans git log . Le dépôt compte 44 branches locales (41 sur origin) et un seul tag (v1.0.0). **Aucune branche release/* n'a jamais existé** : les livraisons passent directement de develop à main par Pull Request (PR #23, #25). Des branches de correction sont également fusionnées directement dans main (PR #24), ce qui est un raccourci par rapport au modèle, pas une application de GitFlow.

Politique de merge.

Point	État
Mécanisme	Pull Request GitHub — main et develop ne reçoivent que des merges, jamais de commit direct dans l'historique récent
Condition technique	Le pipeline GitLab du miroir doit être vert : toutes les portes de lint , test , quality , security , infra , build et package sont bloquantes
Approbations	Non formalisé. Aucune règle de protection de branche n'est versionnée ni documentée, et le dépôt a un seul contributeur : les Pull Requests sont ouvertes et fusionnées par la même personne
Arbitrage d'une exception de sécurité	Passe obligatoirement par la revue de MR, parce qu'une exception n'existe que sous forme d'entrée dans un fichier versionné (AUDIT.md §7.4.4)
Protection de main	Non vérifiable depuis le dépôt. Les réglages de protection vivent dans l'interface GitHub/GitLab et ne sont pas exportés ici

Sur un vrai dépôt d'équipe, deux réglages manquent et ils se posent en cinq minutes : exiger une approbation d'un tiers sur main , et exiger le statut vert du pipeline avant fusion. Ce ne sont pas des oublis de conception, ce sont des réglages d'interface qu'un dépôt à un contributeur ne fait pas ressentir.

3.3 Conventions de nommage

Branches. Le nommage n'est pas cosmétique : le pipeline filtre les jobs sur le **préfixe**, donc une branche mal nommée ne déclenche rien.

- Séparateur **slash**, jamais underscore : la règle est `$CI_COMMIT_BRANCH =~ /^feature\\/.feature_ma-fonctionnalite` ne correspond à aucune règle.
- Mots composés en tiret, en minuscules : `feature/initial-documentation`.
- Préfixes reconnus : `feature/`, `release/`, `hotfix/`, plus `main` et `develop`.

⚠ **Un piège vérifié dans le dépôt.** Les branches `feat/gestion-des-erreurs-http` et `docs/indicateurs-dora-actualises` existent et **ne correspondent à aucune règle de branche** : `feat/` n'est pas `feature/`, et `docs/` n'est prévu nulle part. Poussées seules, elles ne déclenchent aucun job ; elles ne sont vérifiées qu'une fois ouvertes en merge request, où la règle `$CI_MERGE_REQUEST_ID` s'applique. C'est exactement le mode de défaillance que `ARCHITECTURE.md` §7 annonce, observé sur le dépôt lui-même.

Commits. Conventional Commits, imposés par `commitlint` via le hook `.husky/commit-msg`, donc **avant le push** et non en CI. Onze types autorisés (`commitlint.config.js`) :

```
feat · fix · docs · style · refactor · perf · test · build · ci · chore · revert
```

Forme : `type(portée): description à l'impératif`. Exemples réels du dépôt :

```
feat(release): propage la version SemVer jusqu'à l'image et vérifie sa concordance
fix(deploy): échoue si le namespace de destination est vide au lieu de viser default
ci(securite): rend les trois scans Trivy bloquants
```

L'intérêt dépasse le style : un historique exploitable par machine ouvre la génération automatique de changelog et le versionnage sémantique automatisé — deux évolutions identifiées mais **non implémentées** à ce jour (`RELEASE.md` §8).

4. Stratégie de tests automatisés

4.1 Les types de tests

Type de test	Outil	Déclenchement	Couverture cible	État réel
Unitaires	JUnit 5 (back), Karma/Jasmine (front)	Étape <code>test</code> , à chaque push et MR ; filtrés par périmètre sur <code>feature/</code>	≥ 90 % de lignes	97 % lignes / 100 % branches (back) · 100 % lignes / 89 % branches , 112 tests (front)
Intégration	JUnit + Spring Boot, sur PostgreSQL réel (service <code>postgres:16-alpine</code>)	Étape <code>test</code> , jobs <code>test-back</code> et <code>mutation-back</code>	incluse dans le seuil ci-dessus	Repositories, cascades, contrat HTTP de Spring Data REST, CORS, Actuator
E2E	—	—	—	Non implémenté. Aucun stage <code>integration</code> , aucun Cypress ni Playwright. C'est la lacune nommément identifiée en <code>VEILLE.md §8</code> et <code>schema.md</code> : rien ne vérifie que le front et le back fonctionnent ensemble
Sécurité	OWASP Dependency-Check, Trivy (<code>fs</code> et <code>image</code>)	Étape <code>security</code> (dépôt) et fin de <code>package-*</code> (images)	zéro CVE HIGH/CRITICAL non arbitrée	Bloquant depuis le 2026-09-19. Seuil <code>failBuildOnCVSS = 7.0</code> ; les deux images sortent à zéro CVE HIGH/CRITICAL
SonarQube	SonarQube / SonarCloud + SpotBugs + Find-Sec-Bugs	Étape <code>quality</code> , jobs <code>sonar-back</code> , <code>sonar-front</code> , <code>quality-gate</code> , <code>spotbugs-back</code>	Quality Gate franchie sur le code nouveau	Quality Gate opposable (§4.3). <code>spotbugs-back</code> est en <code>ignoreFailures = true</code> : il publie, il ne bloque pas
Performance	k6 (<code>grafana/k6:2.1.0</code> , version figée)	Étape <code>perf</code> , pipeline enfant , après <code>package</code>	p95 lecture < 500 ms, p95 écriture < 800 ms, erreurs < 1 %, p95 smoke < 1 500 ms	<code>k6-smoke</code> bloquant · <code>k6-load</code> en <code>allow_failure</code> (runners mutualisés) · <code>k6-stress</code> sur demande (<code>K6_STRESS=true</code>)

S'y ajoutent deux suites que le template ne prévoit pas et qui pèsent lourd ici, parce que toute la logique du pipeline vit dans des scripts :

Suite	Ce qu'elle couvre	Vérifié
<code>scripts/tests/run_tests.sh</code>	Tous les scripts du pipeline, avec <code>kubect</code> <code>l</code> , <code>docker</code> , <code>trivy</code> remplacés par des faux binaires en tête de <code>PATH</code>	266 assertions , exécuté le 2026-09-24, 0 échec
<code>scripts/tests/validate_k8s.sh</code>	Les manifestes Kustomize et le chart Helm, sans cluster, y compris l'équivalence des deux rendus	96 assertions , exécuté le 2026-09-24, 0 échec

Pourquoi tester des scripts de déploiement. Un script qui échoue mal est plus dangereux qu'un script absent. Les faux `kubect``l` permettent de vérifier les chemins d'échec — rollback automatique déclenché, refus de pousser une image vulnérable — sans cluster ni registry ( `VEILLE.md` §5).

Sur quel moteur de base les tests tournent, et pourquoi ce n'est pas une inconséquence. En CI, un **PostgreSQL réel** démarré comme service du job ; en local, **HSQLDB en mémoire**, sans rien à installer. La bascule ne passe par aucun profil Spring : elle tient aux trois variables `SPRING_DATASOURCE_URL` , `SPRING_DATASOURCE_USERNAME` et `SPRING_DATASOURCE_PASSWORD` . Absentes, HSQLDB ; présentes, PostgreSQL. Le schéma de MicroCRM n'est écrit nulle part — Hibernate le déduit des entités — donc personne ne le relit avant qu'il n'existe, et ce qu'un moteur tolère l'autre le refuse. Une suite verte sur HSQLDB ne dit rien de PostgreSQL (`QUALITY.md` §7).

4.2 Intégration dans le pipeline

Le pipeline compte **10 étapes et 37 jobs**, définis dans `.gitlab-ci.yml` et **14 fichiers** au total (la racine, qui ne contient aucun job, et 13 fichiers de `.gitlab/ci/` , un par domaine).

Étape	Jobs	Ce qu'elle décide	Bloquante
lint	lint-front , lint-back , shellcheck , lint-k8s , lint-helm , version-consistency	Forme du code, manifestes, chart, concordance des versions	oui
test	test-scripts , test-front , test-back	Comportement	oui
quality	sonar-back , sonar-front , spotbugs-back , coverage-gate , mutation-back , quality-gate	Tenue du code, couverture, force des assertions	oui, sauf spotbugs-back
security	dependency-check-back , trivy-fs	Surface d'attaque du dépôt	oui
infra	terraform-validate , terraform-plan , ansible-lint	L'infrastructure se valide avant qu'on ne compile	oui
build	build-front , build-back	Artefacts	oui
package	package-back , package-front , promote-back , promote-front	Images taguées par SHA, scannées, poussées	oui
perf	perf → pipeline enfant (k6-smoke , k6-load , k6-stress)	Tenue en charge de l'image construite	k6-smoke seul
deploy	deploy-staging , deploy-production , rollback-production	Mise en service et retour arrière	manuels
infra-apply	terraform-apply-{staging,logging,production} , notify-echec	terraform apply par environnement, notification d'échec	manuels, bloquants

Trois choix d'ordonnancement méritent leur explication.

1. **security avant build** . C'est le shift-left rendu littéral : on ne compile pas ce dont on sait déjà qu'il ne sera pas livrable.
2. **infra avant build** . Un plan Terraform ou un manifeste cassé n'a pas besoin d'attendre une compilation Gradle pour être signalé.
3. **infra-apply en dernier, et c'est un correctif, pas un rangement**. Un job manuel **sans** `allow_failure` est un job bloquant : « the pipeline stops at the stage where the job is defined ». Tant que les trois `terraform-apply-*` vivaient dans l'étape `infra` , tout pipeline de `main` s'y arrêtrait, et `build` , `package` , `perf` et `deploy` n'étaient jamais atteints tant que personne ne cliquait.

Conditions de déclenchement.

Régime	Ce qui tourne
Merge request	Toute la vérification, filtrée par périmètre : un commit qui ne touche que <code>front/</code> ne relance pas <code>test-back</code> , <code>mutation-back</code> ni <code>dependency-check-back</code>
Branche <code>feature/...</code>	Idem, avec <code>compare_to: refs/heads/develop</code> pour que le filtre réponde « ce que cette branche change », pas « ce que ce push change »
<code>develop</code> , <code>main</code> , <code>release/</code> , <code>hotfix/</code>	Tout, sans filtre de périmètre — avant une livraison, on ne saute rien
Tag <code>vX.Y.Z</code>	Tout sauf <code>package-*</code> : <code>promote-back</code> et <code>promote-front</code> retaguent l'image déjà publiée

Ce que le filtrage n'ose pas franchir. `build-*`, `package-*` et les jobs de déploiement gardent leurs règles **sans** `changes:`. Les images sont taguées par `$CI_COMMIT_SHORT_SHA`, et `deploy` comme les services k6 les réclament sous ce tag : sauter `package-back` parce que le back n'a pas bougé produirait un tag qui n'existe pas. Le filtrage s'arrête à la vérification.

4.3 Quality Gate SonarQube

Le principe : *Clean as You Code*. La porte ne s'applique pas à l'ensemble du code existant — exigence décourageante et jamais atteinte — mais au **code nouveau ou modifié**. La dette se résorbe à mesure que le code est touché.

Comment le verdict est récupéré. Les jobs `sonar-back` et `sonar-front` envoient l'analyse ; ils ne lisent pas le verdict. C'est un job séparé, `quality-gate`, qui appelle `scripts/ci/quality_gate.py` : le script interroge l'API jusqu'à obtenir le résultat de l'analyse, affiche les règles fautives, et **sort en 2** si la porte n'est pas franchie. Il est lancé deux fois, une par projet Sonar.

Les critères effectivement opposables, tous contrôlés dans l'étape `quality` :

Critère	Seuil	Job	Où le seuil est écrit
Quality Gate Sonar sur le code neuf	verdict <code>OK</code>	<code>quality-gate</code>	Côté serveur Sonar
Couverture de lignes (back)	90 % (<code>COVERAGE_MIN</code>)	<code>coverage-gate</code>	<code>.gitlab/ci/variables.yml</code> , appliqué par <code>scripts/ci/check_coverage.py</code>
Couverture de branches (back)	90 %	<code>./gradlew check</code> en local	<code>jacocoTestCoverageVerification</code> dans <code>back/build.gradle</code>
Couverture (front)	90 % lignes / 80 % branches	<code>test-front</code>	bloc <code>check.global</code> de <code>front/karma.conf.js</code>
Score de mutation (back)	80 % (<code>MUTATION_MIN</code>)	<code>mutation-back</code>	<code>.gitlab/ci/variables.yml</code> et <code>back/build.gradle</code>

Pourquoi deux seuils qui ne mesurent pas la même chose. La couverture dit quelles lignes les tests traversent ; **PIT dit si les tests vérifient quelque chose.** Un test sans aucune assertion affiche 100 % de couverture. PIT modifie le bytecode — inverse une condition, remplace un retour par `null`, supprime un appel — puis relance les tests qui couvrent la ligne mutée. Si aucun ne devient rouge, le mutant survit : la ligne est exécutée, mais rien ne l'observe.

Action en cas d'échec.

1. **Le pipeline s'arrête.** `quality-gate`, `coverage-gate` et `mutation-back` sont bloquants : rien n'est construit, rien n'est packagé, rien n'est déployable. Le seul contrôle de l'étape `quality` qui ne bloque pas est `spotbugs-back` (`ignoreFailures = true`), qui publie ses findings en artefact.
2. **La correction se fait dans la branche**, pas dans le seuil. Les seuils sont calés **sous** les valeurs tenues, avec assez de marge pour ne pas se déclencher sur une ligne de plus et assez peu pour qu'une vraie régression se voie. Un seuil qu'on abaisse à la première gêne ne protège plus rien.
3. **Le doublon local est voulu.** `jacocoTestCoverageVerification` rejoue le seuil dans `./gradlew check` : un seuil qui ne se déclenche qu'en CI se découvre toujours après le push.
4. **Une seule exclusion, nommée** : `MicroCRMApplication`, dont la méthode `main` ne contient que l'amorçage Spring Boot — pas un motif large qui absorberait du code métier au passage.

5. Architecture IaC et Cloud

5.1 Vue d'ensemble

Il n'y a pas de fournisseur cloud, et c'est une décision, pas un manque.

Le brief propose AWS ou Azure et **autorise nommément l'environnement local**, y compris pour Terraform. La décision D2 a retenu le tout-local : **minikube v1.38.1, Kubernetes v1.35.1, un nœud, Docker Desktop**, aucun compte, aucune ressource payante. Les raisons, développées dans `ARCHITECTURE.md §9` :

1. **Aucun coût, donc aucune ressource oubliée.** Le mode de défaillance le plus banal d'un projet d'école en cloud n'est pas technique : c'est un compte qui facture parce qu'une ressource a survécu au projet — ou l'inverse, une infrastructure supprimée trop tôt qui rend la démonstration impossible.
2. **L'environnement existait déjà**, addons `ingress` et `registry` compris. Le temps épargné a servi à ce que le projet devait démontrer.
3. **Le livrable reste vérifiable par un tiers, indéfiniment.** Un cluster cloud éteint après la démonstration ne se constate plus ; un dépôt qui se rejoue se constate toujours.
4. **Le local n'est pas une simulation.** Le provider `hashicorp/kubernetes` parle à un vrai serveur d'API et crée de vraies ressources — 6 par environnement. Il n'y a ni plan simulé, ni dry-run présenté comme une preuve.

5. **Gratuit ne veut pas dire sans limite, et ça s'est vérifié.** Pendant deux mois, les pipelines ne s'arrêtaient pas sur un test rouge mais sur `ci_quota_exceeded`. La sortie a été d'enregistrer un **runner auto-hébergé** sur le poste le 2026-09-22. La contrainte de ressource n'a pas disparu : elle s'est déplacée vers la machine.

Type d'architecture. Microservices au sens faible : **deux services** (front et back) déployés séparément, chacun avec son image, son Deployment, son Service et sa règle d'Ingress. Pas de service mesh, pas de passerelle applicative, pas de file de messages.

Services « managés » — aucun. La table de transposition complète, brique par brique, figure dans `ARCHITECTURE.md §10`. En résumé :

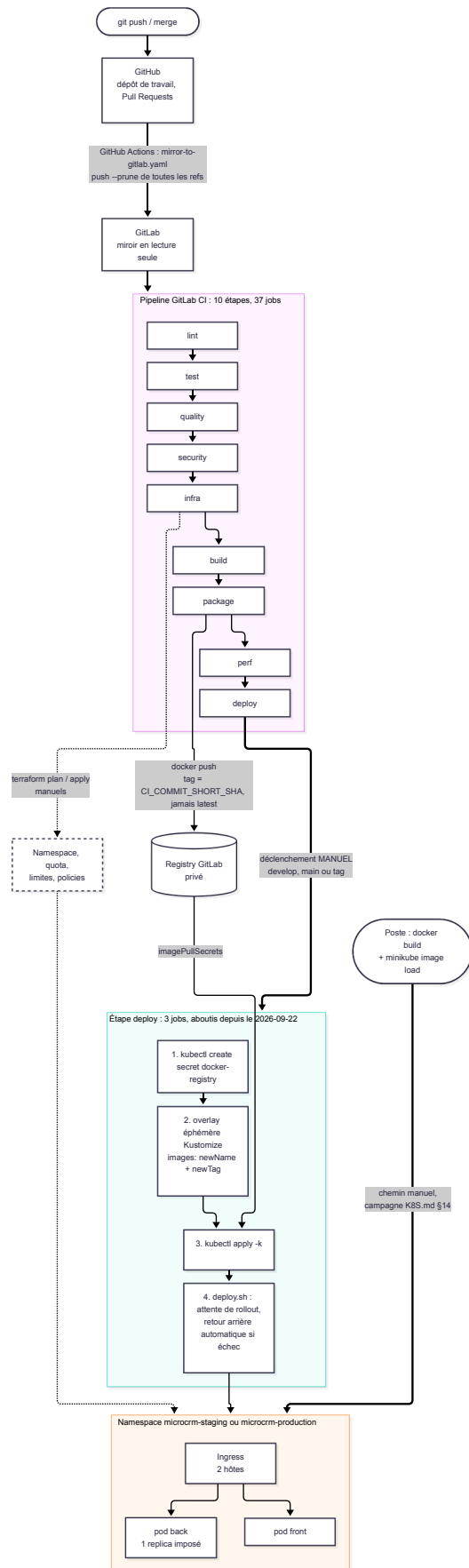
Brique	Aujourd'hui (local)	AWS	Azure
Le cluster	minikube, 1 nœud, créé par Ansible	EKS	AKS
<code>Namespace</code> , <code>ResourceQuota</code> , <code>LimitRange</code>	objets K8s créés par Terraform	inchangés	inchangés
<code>Deployment</code> , <code>Service</code> , <code>ConfigMap</code>	Kustomize, overlays par environnement	inchangés	inchangés
Exposition publique, DNS, TLS	aucune — <code>kubectl port-forward</code> et en-tête <code>Host:</code>	Route 53 + ACM	Azure DNS + Key Vault
Registry	registry GitLab pour la CI, <code>minikube image load</code> en local	ECR	ACR
<code>Secret</code> du registry	recréé par la CI à chaque déploiement	disparaît (IRSA)	disparaît (identité managée)
État Terraform	backend <code>http</code> — état managé GitLab, un par env, verrouillé	S3 + verrouillage	Azure Storage + lease
Stack de logs	Elasticsearch, Kibana, Filebeat dans le namespace <code>logging</code>	OpenSearch / CloudWatch	Log Analytics / Azure Monitor

Ce que le local ne démontre pas, et qu'aucune phrase ne remplace (`ARCHITECTURE.md §11`) : la haute disponibilité (un seul nœud), un LoadBalancer réel, le stockage managé et sauvegardé, l'IAM et la gestion fine des droits, les coûts et leur maîtrise, la montée en charge automatique. Le cloisonnement réseau lui-même est **décrit, pas prouvé** : les `NetworkPolicy` existent et sont correctes, mais le CNI par défaut de minikube ne les implémente pas et les **ignore sans rien signaler**. Un `apply` réussi ne permet donc pas de conclure que le namespace est cloisonné.

5.2 Les schémas

5.2.1 Schéma d'infrastructure

Du commit au pod : la plateforme réelle, registry et namespaces compris. Le trait plein marque un chemin réellement emprunté, le pointillé un chemin décrit dont l'exécution reste manuelle.

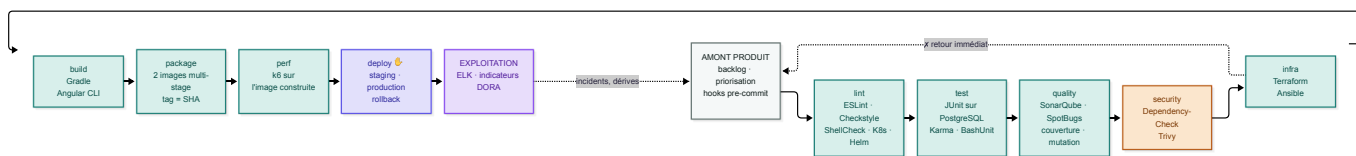


Source : `docs/schemas/plateforme-deploiement.mmd` (version de `develop`), avec le seul libellé du nombre de jobs porté de 36 à 37 depuis l'ajout de `notify-echec`. Rendu vérifié avec `mermaid-cli`.

Ce que le trait plein de l'étape `deploy` veut dire, depuis peu. Ce cadre était en tirets — écrit, testé, jamais mené à son terme. Il ne l'est plus depuis le 2026-09-22. Le runner auto-hébergé a supprimé la contrainte de quota et rendu visibles trois défauts qu'aucun job n'allait assez loin pour rencontrer : un kubeconfig pointant sur `127.0.0.1` depuis un conteneur, un RBAC d'agent qui ne couvrait pas les objets applicatifs, et un `$STAGING_NAMESPACE` absent qui faisait déployer dans `default` en silence. Les trois sont corrigés.

5.2.2 Schéma du pipeline CI/CD

La chaîne complète, de l'amont produit à l'exploitation. La sécurité s'exécute **avant** la construction des artefacts : c'est le shift-left, et c'est ce que la disposition doit rendre lisible.



Source : `docs/schemas/chaine-cicd.mmd` , reprise sans modification. Rendu vérifié avec `mermaid-cli` .

Couleur	Nature
Gris	Amont produit (hors CI)
Vert d'eau	Vérification et livraison
Orange	Sécurité — shift-left / DevSecOps
Indigo	Déploiement
Violet	Exploitation

5.3 Les environnements

Environnement	Branche(s)	Type de déploiement	URL	Particularités
Development	—	Non implémenté	<code>http://localhost:4200</code> (front) et <code>:8080</code> (API) via <code>docker compose up</code>	Il n'existe aucun environnement de développement déployé. Le développement se fait sur le poste, avec Docker Compose ou <code>ng serve</code> ; aucun namespace, aucune cible de déploiement, aucun job de CI ne le vise
Staging	<code>develop</code>	Kubernetes, manuel (when: manual)	<code>http://microcrm.staging.example.com</code> et <code>http://api.microcrm.staging.example.com</code> — hôtes fictifs, résolus nulle part	Namespace <code>microcrm-staging</code> . Accès de test par <code>kubectll port-forward</code> vers le contrôleur d'Ingress, avec l'hôte en en-tête <code>Host:</code> . Le passage en <code>on_success</code> est une ligne (<code>RELEASE.md §6</code>)
Production	<code>main</code> ou tag <code>vX.Y.Z</code>	Kubernetes, manuel — garde-fou volontaire	<code>http://microcrm.example.com</code> et <code>http://api.microcrm.example.com</code> — hôtes fictifs	Namespace <code>microcrm-production</code> . Vise le même minikube dans un autre namespace : il démontre qu'un second environnement

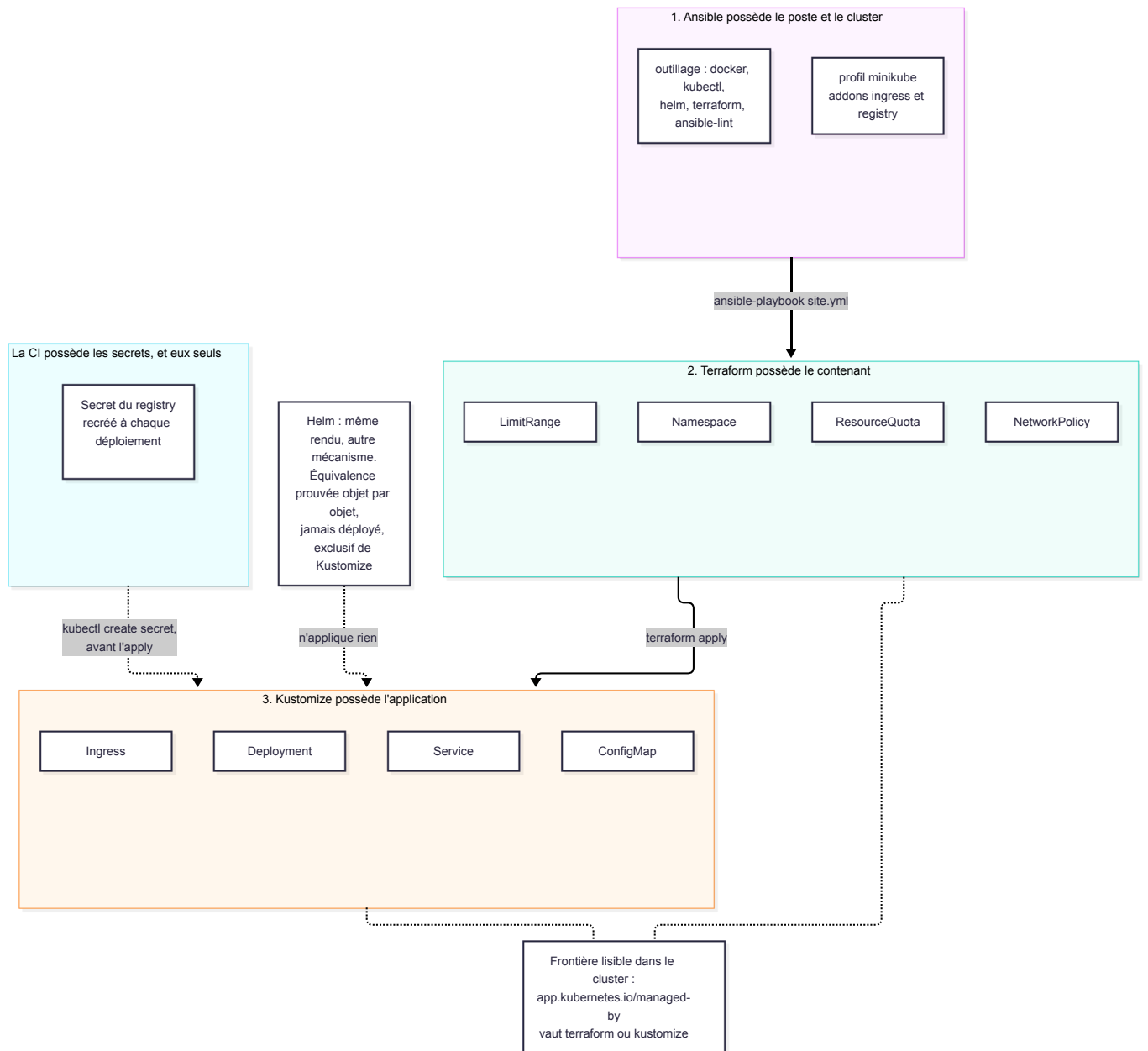
Environnement	Branche(s)	Type de déploiement	URL	Particularités
				se décrit par les mêmes modules et d'autres valeurs, pas qu'une production existe

Trois précisions qui changent la lecture de ce tableau.

- **Aucune URL n'est jointe depuis l'extérieur.** Il n'y a ni `Service` de type `LoadBalancer`, ni adresse publique, ni DNS, ni certificat. Les hôtes d'Ingress sont là pour que le routage soit décrit et vérifiable, pas pour être résolus.
- **Le namespace vient d'une variable, jamais du manifeste** (`$STAGING_NAMESPACE`, `$PROD_NAMESPACE`). Le 2026-09-22, l'une d'elles n'était pas définie : `kubectl apply -n ""` est retombé sur `default` **sans rien signaler**, et l'application a été déployée au mauvais endroit. Le garde-fou `exige_namespace` fait désormais échouer le job. Il ne couvre toujours pas la divergence entre deux noms tous les deux renseignés, qui reste à la charge de la relecture.
- **Un quatrième namespace existe, hors application** : `logging`, créé par Terraform comme tout autre contenant, qui héberge Elasticsearch, Kibana et Filebeat.

5.4 Structure du code laC

La règle centrale, celle qui décide de tout le reste : **un objet a un propriétaire, et un seul**. Deux outils qui écrivent la même ressource produisent un `terraform plan` qui propose sans fin de défaire ce que le dernier `kubectl apply` a posé.



Source : `docs/schemas/iac-frontiere.mmd`, reprise sans modification. Rendu vérifié avec `mermaid-cli`.

L'arborescence réelle.

```
graph LR
    terraform["terraform/"]
    terraform --> modules["modules/"]
    modules --> namespace["namespace/"]
    modules --> network_policy["network-policy/"]
    modules --> environments["environments/"]
    namespace --> namespace_desc["namespace + ResourceQuota + LimitRange"]
    network_policy --> network_policy_desc["default-deny + deux autorisations"]
    environments --> staging["staging/"]
    environments --> production["production/"]
    environments --> logging["logging/"]
    staging --> staging_desc["versions.tf · providers.tf · variables.tf  
terraform.tfvars ← la seule chose qui distingue  
main.tf · outputs.tf"]
    production --> production_desc["mêmes fichiers, autres valeurs"]
    logging --> logging_desc["la stack ELK, même modèle"]

    ansible["ansible/"]
    ansible --> site_yaml["site.yml"]
    ansible --> inventory["inventory/local.yml"]
    ansible --> group_vars["group_vars/all.yml"]
    ansible --> roles["roles/"]
    site_yaml --> site_desc["le point d'entrée"]
    inventory --> inventory_desc["le poste, et lui seul"]
    roles --> outillage["outillage/"]
    roles --> cluster["cluster/"]
    outillage --> outillage_desc["docker, kubectl, helm, terraform, ansible-lint"]
    cluster --> cluster_desc["profil minikube, addons ingress et registry"]

    k8s["k8s/"]
    k8s --> base["base/"]
    k8s --> overlays_staging["overlays/staging/"]
    k8s --> overlays_production["overlays/production/"]
    k8s --> elk["elk/"]
    base --> base_desc["Deployment ×2, Service ×2, Ingress, ConfigMap"]
    overlays_staging --> overlays_staging_desc["patches de ConfigMap et d'Ingress"]
    overlays_production --> overlays_production_desc["+ patch de ressources du back"]
    elk --> elk_desc["Elasticsearch, Kibana, Filebeat, RBAC, dashboards"]

    helm_microcrm["helm/microcrm/"]
    helm_microcrm --> helm_microcrm_desc["chart équivalent, values par environnement"]
```

Quatre règles de structure, et ce qu'elles achètent.

1. **Les environnements ne déclarent aucune ressource, ils composent des modules.** C'est ce qui garantit que staging et production ne peuvent pas diverger autrement que par leurs valeurs — sans quoi le premier cesserait de valider quoi que ce soit du second.
2. **`terraform.tfvars` est le seul fichier qui distingue deux environnements.** Si une différence de comportement ne se lit pas là, c'est qu'elle s'est glissée ailleurs.
3. **`.terraform.lock.hcl` est versionné**, pour les quatre plateformes qui exécutent le projet. Un lock qui ne couvre que le poste se laisse compléter en CI, donc ne verrouille rien.
4. **Les manifestes ne contiennent ni namespace ni chemin de registry.** Ces coordonnées viennent de la CI, via un **overlay éphémère** fabriqué au moment du déploiement. C'est ce qui permet à `lint-k8s` de valider les manifestes sans coordonnées, et c'est aussi ce qui rend les manifestes portables tels quels vers un autre fournisseur.

L'état Terraform. Backend `http`, état managé par GitLab, **un par environnement et verrouillé**. Le chemin a été éprouvé en lecture et en écriture le 2026-09-22, lors de la migration d'un état local vers ce backend : verrou pris et relâché, état écrit, état relu, `terraform plan` répondant « No changes » (`RELEASE.md` §9.5). Il reste à le jouer **depuis un job de CI**, avec `$CI_JOB_TOKEN` au lieu d'un jeton personnel.

⚠ **La couture que le schéma ne montre pas, parce qu'elle n'existe nulle part.** Le nom du namespace est écrit à deux endroits — le `terraform.tfvars` de l'environnement et la variable GitLab — et **rien ne compare les deux**. Terraform crée le namespace, la CI y déploie, et les deux ne se parlent pas.

6. Sécurité et qualité du code

6.1 Approche DevSecOps / shift-left

Une vulnérabilité détectée à l'écriture coûte quelques minutes ; détectée en production, elle coûte un incident. Chaque contrôle est donc placé au plus tôt dans la chaîne : les hooks `husky` filtrent avant le push, l'étape `security` s'exécute **avant** `build` — on ne compile pas ce dont on sait déjà qu'il ne sera pas livrable — et le scan d'image est accolé à sa construction, dans `package-*`, pas dans un job de fin de pipeline qu'on lirait après coup.

Le deuxième pilier est qu'un contrôle qui signale sans arrêter finit par être lu comme du bruit. Les quatre portes de sécurité étaient en `allow_failure` et ne décidaient de rien ; elles sont bloquantes depuis le **2026-09-19**. C'est ce qui transforme une intention en processus (§6.4).

6.2 Les outils d'analyse

Outil	Type d'analyse	Rôle	Étape	Bloquant
SonarQube / SonarCloud	SAST + dette technique	Bonnes pratiques, bugs, duplication, complexité, code mort ; agrège la couverture back et front ; porte la Quality Gate	quality	oui (<code>quality-gate</code>)
SpotBugs + Find-Sec-Bugs	SAST sur le bytecode	Ce qu'aucun linter de texte ne voit : déréréfencement null sur un chemin précis, comparaison de <code>String</code> avec <code>==</code> , flux non fermés. Find-Sec-Bugs ajoute ~140 patterns de sécurité (injection, XSS, CORS permissif, crypto faible)	quality	non — <code>ignoreFailures = true</code> , publie en artefact
ESLint, Checkstyle, Spotless, Prettier, ShellCheck	Linting et mise en forme	Forme du code TypeScript, Java et Bash ; les deux premiers aussi en local via <code>lint-staged</code>	lint + hooks pre-commit	oui
<code>lint-k8s</code> , <code>lint-helm</code>	Linting d'infrastructure	Rendu des overlays Kustomize et du chart Helm, et leur équivalence objet par objet	lint	oui
OWASP Dependency-Check	SCA	Confronte l'arbre de dépendances Gradle à la base NVD. C'est le contrôle qui aurait détecté Log4Shell	security	oui , <code>failBuildOnCVSS = 7.0</code>
Trivy (<code>image</code>)	Container scanning	CVE des couches système de l'image finale : Alpine, JRE, Caddy — chacun avec ses propres CVE	fin de <code>package-back</code> / <code>package-front</code>	oui , <code>--exit-code 1</code> sur HIGH,CRITICAL
Trivy (<code>fs</code>)	Secrets detection + misconfig + CVE du dépôt	Secrets commités, mauvaises configurations de Dockerfile et de manifestes	security	oui , avec <code>--ignorefile .trivyignore.yaml</code>

Trois lacunes de couverture, nommées plutôt que tuées.

- **Aucun `npm audit`**. Il n'existe pas d'équivalent de Dependency-Check côté front : les dépendances npm ne sont couvertes que par `trivy-fs`, qui lit le `package-lock.json`. C'est une piste identifiée, pas un contrôle en place.

- **spotbugs-back ne bloque pas.** Le flag se passe à `false` dans `back/build.gradle` le jour où l'équipe le décide.
- **Aucune signature d'image.** Les images sont taguées par SHA, pas signées ; il n'y a pas d'attestation de provenance (`schema.md`).

Ce que les scans trouvent aujourd'hui. Le scan du dépôt sort en 0 une fois les exclusions appliquées, et les deux images ne portent plus aucune CVE HIGH ou CRITICAL depuis la montée de version du 2026-09-19 (Alpine 3.24, Spring Boot 3.5.16, Caddy recompilé). **La porte se ferme donc sur du vide, et c'est voulu : ce qui arrêtera le pipeline, c'est ce qui sera introduit après.**

6.3 Gestion des secrets

Le principe : un secret ne descend jamais dans le dépôt, et l'historique Git ne pardonne pas. Un secret engagé y reste pour toujours, même après correction.

Secret	Où il vit	Comment il atteint le job
<code>SONAR_TOKEN</code>	Variable CI/CD GitLab, masquée	Variable d'environnement du job
<code>NVD_API_KEY</code>	Variable CI/CD GitLab, masquée (facultative)	Variable d'environnement
<code>NOTIFY_WEBHOOK_URL</code>	Variable CI/CD GitLab, masquée (facultative)	Lue par <code>scripts/ci/notify.py</code>
<code>CI_REGISTRY_USER</code> / <code>_PASSWORD</code>	Fournies automatiquement par GitLab	Utilisées pour créer le <code>Secret</code> Kubernetes du registry
Accès au cluster	Agent Kubernetes GitLab , plus aucune variable	Tunnel sortant de l'agent ; <code>kubectl config use-context "\$CI_PROJECT_PATH:\$KUBE_AGENT_NAME"</code>
<code>GITLAB_TOKEN</code> (miroir)	Secret GitHub Actions, scope <code>write_repository</code>	Workflow <code>mirror-to-gitlab.yaml</code>
Secret du registry dans K8s	Jamais versionné , recréé à chaque déploiement	<code>kubectl create secret docker-registry ... --dry-run=client -o yaml kubectl apply -f -</code>

Trois décisions valent d'être détaillées.

1. **Un `Secret` Kubernetes n'est pas chiffré.** Son champ `data` est du base64, un encodage et non une protection. Le commiter reviendrait à publier le mot de passe du registry. Il est donc la troisième valeur — après le namespace et le chemin du registry — que les manifestes **réfèrent sans la contenir**.
2. **Le mot de passe ne doit jamais atteindre le journal, et c'est contre-intuitif.** La sortie de `-o yaml` contient le mot de passe encodé en base64, or **le masquage GitLab travaille sur la valeur littérale** et ne reconnaît pas cette forme encodée. Le flux va donc directement dans le tube : pas de `tee`, pas

de `cat`, et surtout pas de `kubect1 get secret -o yaml` ajouté « pour vérifier ». Un commentaire le rappelle à l'endroit exact où la tentation se présente.

3. **KUBE_CONFIG a disparu le 2026-09-23, et ce n'est pas un oubli.** Un kubeconfig de poste désigne le serveur d'API en `127.0.0.1`, ce qui est structurellement injoignable depuis un conteneur de job. Les jobs de déploiement passent désormais par l'agent Kubernetes, comme les jobs Terraform.
4. **Ce qui n'est pas un secret ne doit pas être traité comme tel.** Les identifiants du PostgreSQL de test sont écrits en clair dans le pipeline : la base naît et meurt avec le job, n'est joignable que depuis son réseau, et ne contient que ce que les tests y écrivent. En faire une variable masquée laisserait croire à un secret là où il n'y en a pas — et le vrai risque des secrets est qu'on cesse de distinguer ceux qui en sont.
5. **terraform.tfstate ne contiendra jamais le Secret du registry.** L'état contient en clair tout ce que les providers ont lu, y compris les attributs `sensitive` : le masquage ne vaut que pour l'affichage. C'est l'une des raisons de la frontière du §5.4.

Politique de rotation — non formalisée

Il n'existe aucune politique de rotation des secrets dans ce projet. Ni périodicité, ni procédure, ni inventaire des porteurs. L'information a été cherchée dans `VARIABILISATION.md`, `QUALITY.md`, `K8S.md`, `AUDIT.md` §7 et `RELEASE.md` : aucune ne la mentionne.

Deux faits atténuent partiellement le risque, sans tenir lieu de politique : les identifiants du registry sont **fournis et renouvelés par GitLab à chaque job** (`CI_REGISTRY_PASSWORD` est un jeton de job, à durée de vie limitée à celle du job), et le `Secret` Kubernetes correspondant est **recréé à chaque déploiement** par une commande idempotente qui l'actualise si le mot de passe a changé. Les secrets réellement statiques sont `SONAR_TOKEN`, `NVD_API_KEY`, `NOTIFY_WEBHOOK_URL` et le `GITLAB_TOKEN` du miroir : **rien n'impose ni ne trace leur renouvellement.**

Ce qu'une politique devrait fixer, et qui reste à écrire : une périodicité par type de secret, un porteur nommé, une procédure de révocation immédiate en cas de suspicion, et une trace datée du dernier renouvellement.

6.4 Processus de traitement des vulnérabilités

Ce processus n'est pas une intention : il est imposé par la chaîne depuis le 2026-09-19, date à laquelle les quatre portes sont devenues bloquantes.

Ce qui déclenche

Porte	Ce qu'elle voit	Seuil	Étape
<code>dependency-check-back</code>	CVE des dépendances Java	CVSS ≥ 7	<code>security</code>
<code>trivy-fs</code>	Secrets commités, misconfigurations, CVE du dépôt	HIGH, CRITICAL	<code>security</code>
<code>package-back</code> / <code>package-front</code>	CVE des couches système de l'image	HIGH, CRITICAL	<code>package</code>

Le seuil de Dependency-Check est `failBuildOnCVSS = 7` (`back/build.gradle`), soit la borne basse de la sévérité HIGH du CVSS v3 : les deux familles d'outils s'arrêtent au même niveau de gravité, ce qui évite qu'une CVE bloque d'un côté et passe de l'autre.

Les délais selon la criticité

Criticité	Délai de traitement	Ce qui l'impose
CRITICAL (CVSS ≥ 9,0)	Zéro — immédiat, par construction	Porte bloquante : rien n'est livré tant que la découverte n'a pas été arbitrée
HIGH (CVSS 7,0 – 8,9)	Zéro — immédiat, par construction	Même porte, même seuil
MEDIUM (CVSS 4,0 – 6,9)	Aucun délai défini	Sous le seuil des portes : ces CVE ne sont pas remontées par les jobs, donc pas suivies
LOW	Aucun délai défini	Idem

Ce tableau dit deux choses, et la seconde est la plus importante. Au-dessus du seuil, le délai est nul et il n'y a pas d'arbitrage entre traiter et remettre à plus tard : c'est tout l'intérêt d'avoir retiré les `allow_failure`. **En dessous du seuil, il n'y a rien** — ni suivi, ni inventaire, ni échéance. Fixer le seuil à 7 est un choix cohérent entre les deux familles d'outils ; il laisse néanmoins les CVE MEDIUM hors de toute gestion, et ce n'est écrit nulle part ailleurs que dans ce paragraphe.

La contrepartie du délai nul est assumée : une CVE publiée dans une dépendance transitive peut bloquer une livraison sans rapport avec le changement en cours. C'est le prix, et il est préférable à une livraison qui ignore ce qu'elle emporte.

Les trois issues, dont une seule est la voie normale

1. **Corriger** — monter la version de la dépendance ou de l'image de base. C'est la voie par défaut, et celle qui doit être tentée en premier.
2. **Inscrire une exception motivée** — uniquement quand la vulnérabilité ne s'applique pas au contexte, ou quand aucun correctif n'existe et que le risque résiduel est acceptable **et écrit**.
3. **Arrêter la livraison** — quand ni l'un ni l'autre n'est possible. Ne rien livrer reste une décision valable.

Exemple vécu, issue n° 1. Le 2026-09-19, Trivy a bloqué sur des CVE des couches système. La réponse a été de monter Spring Boot de 3.2.5 à 3.5.16 et de recompiler Caddy — commit `df1634f` — et non d'inscrire une exception.

Qui arbitre, et où s'écrit une exception

L'arbitrage passe par la revue de merge request, jamais par une décision individuelle. C'est une conséquence du format retenu : une exception n'existe que sous la forme d'une entrée dans un fichier versionné. Elle arrive donc dans une MR, avec sa justification, et ne peut pas être posée en silence par la personne que le pipeline dérange.

Registre	Ce qu'il couvre	État actuel
<code>.trivyignore.yaml</code>	Misconfigurations et CVE d'image	4 entrées , revue au 2026-12-31
<code>back/config/dependency-check/suppressions.xml</code>	CVE de dépendances (faux positifs)	vide
<code>.trivyignore</code> (format historique)	—	conservé vide délibérément

Trois règles de forme, et elles ne sont pas décoratives :

- **Une exception est bornée à un chemin.** Le format historique `.trivyignore` applique un identifiant à tout le dépôt : ignorer `KSV-0014` pour un manifeste masquerait le jour où un autre perdrait vraiment son `readOnlyRootFilesystem`. D'où le fichier vide et le `--ignorefile .trivyignore.yaml` explicite.
- **Une exception porte une justification écrite**, qui dit pourquoi la vulnérabilité ne s'applique pas — pas qu'elle dérange.
- **Une exception porte une date de revue** (`expiredAt`). Le fichier est relu à chaque release, et **une entrée expirée redevient bloquante d'elle-même**.

Politique de mise à jour des dépendances

Trois registres se mettent à jour séparément, et les confondre est la première cause de mise à jour oubliée :

Registre	Où	Contrôlé par
Dépendances applicatives	<code>back/build.gradle</code> , <code>front/package.json</code>	<code>dependency-check-back</code>
Images de base	<code>back/Dockerfile</code> , <code>front/Dockerfile</code>	Trivy, à l'étape <code>package</code>
Images d'outillage de CI	<code>.gitlab/ci/variables.yml</code>	aucun contrôle automatique

Règle commune : aucune version flottante. Ni `latest` , ni plage ouverte. Une montée de version se fait dans un **commit dédié qui dit pourquoi**.

Cadence : à chaque pipeline (automatique, c'est le mécanisme principal), à chaque release (relecture des exceptions dont la date approche), et sur publication d'une CVE majeure sans attendre le pipeline suivant.

Ce que ce processus ne couvre pas encore

- **Le troisième registre n'est surveillé par rien.** Les images d'outillage de la CI sont figées — la bonne décision — mais aucun contrôle ne signale qu'une version figée a vieilli. Elles ne tournent pas en production, donc le risque est indirect ; il n'est pas nul, puisqu'elles manipulent les identifiants du registry.
- **Aucun outil de mise à jour automatique.** Ni Dependabot, ni Renovate. Ce n'est pas un choix argumenté, c'est une absence. Renovate est l'option adaptée, parce qu'il couvre Gradle, npm **et** les images Docker d'un fichier CI GitLab — les trois registres, là où Dependabot ignorerait le troisième.
- **Un écart entre Dependency-Check et Trivy n'a jamais été expliqué.** Dependency-Check ne remontait pas des CVE Tomcat que Trivy voyait. Les deux outils regardent des périmètres différents — dépendances déclarées pour l'un, couches d'image pour l'autre — ce qui explique peut-être tout, mais ne l'a pas été.

7. Automatisation des releases et rollback

7.1 Stratégie de déploiement

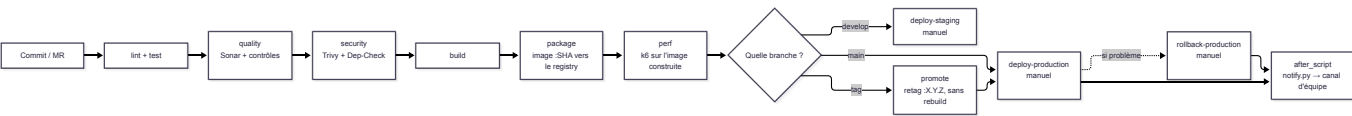
Rolling update, avec `maxSurge: 1` et `maxUnavailable: 0`. Ni blue/green, ni canary.

La justification tient en trois points, et le premier est une contrainte, pas une préférence.

1. **La base de MicroCRM vit en mémoire (HSQLDB), donc le back est plafonné à un replica.** Un blue/green ou un canary supposent deux versions servant du trafic en même temps ; avec deux pods back, on aurait deux bases distinctes et une requête sur deux ne verrait pas ce que l'autre a écrit, **sans qu'aucune erreur ne soit levée**. La stratégie progressive n'est donc pas coûteuse ici : elle est structurellement impossible tant que l'état n'est pas sorti du processus.
2. `maxUnavailable: 0` **impose que le nouveau pod soit prêt avant que l'ancien ne parte.** C'est ce qui donne une mise à jour sans interruption perceptible — et c'est aussi ce qui oblige à dimensionner le `ResourceQuota` sur le **pic d'un déploiement**, pas sur le régime permanent, sous peine de voir le déploiement suivant bloqué sur un `exceeded quota` qui ne dit pas qu'il s'agit d'un problème de dimensionnement.
3. **Kubernetes conserve nativement l'historique des révisions.** C'est l'argument qui a fait retenir Kubernetes plutôt que Docker Compose : le rollback devient une commande au lieu d'une procédure d'urgence improvisée (`VEILLE.md` §4).

Le déploiement progressif est donc « non implémenté », avec sa raison. Il figure au plan sous l'action A4.1 (`docs/plan-optimisation-release.md` §4), en vague 4, et **après** la vague 2 qui sort la base du processus : un canary sur une base non persistante ne prouverait rien.

7.2 Pipeline de release — du tag jusqu'à la notification



Les sept étapes d'une mise en production (`RELEASE.md` §7) :

1. **Merger sur `main`** via une MR au pipeline vert. **C'est ce pipeline qui construit l'image** et la pousse sous le SHA du commit.
2. **Créer le tag** : `git tag vX.Y.Z && git push origin vX.Y.Z`.
3. **`version-consistency` s'exécute dès la première étape** du pipeline de tag, avant qu'on ait construit ou scanné quoi que ce soit.
4. **`promote-back` et `promote-front` retaguent** l'image déjà publiée pour ce commit. Si l'étape 1 n'a pas abouti, le `docker pull` échoue et le job avec — c'est voulu : on ne publie pas un numéro de version qui ne désigne aucun artefact.
5. **Lancer `deploy-production` à la main** et valider. Le job résout `DEPLOY_IMAGE_TAG` = le numéro de version sur un pipeline de tag, le SHA partout ailleurs, puis pose ce tag dans l'overlay éphémère.
6. **`deploy.sh` attend le rollout** et revient de lui-même en arrière s'il n'aboutit pas dans `$DEPLOY_TIMEOUT`.
7. **Notification.** L' `after_script` de `.deploy_template` appelle `scripts/ci/notify.py` avec `$CI_JOB_STATUS` — donc **dans les deux cas**, succès comme échec.

Un pipeline de tag ne reconstruit rien, et c'est le cœur du dispositif. `package-back` et `package-front` n'y tournent pas (`.rules_package`). Deux builds du même commit ne produisent pas les mêmes couches — horodatages, résolution de paquets — donc une image reconstruite au moment du tag aurait les mêmes sources mais ne serait plus celle que Trivy a scannée ni celle que k6 a mise sous charge. **En retaguant, `1.4.0` est l'artefact éprouvé, pas un jumeau.**

7.3 Système de versioning

La source de vérité est le tag Git, au format **SemVer** : `git tag v1.4.0` déclare la version.

Trois fichiers la répètent — `front/package.json`, `back/build.gradle` et l' `appVersion` de `helm/microcrm/Chart.yaml` — pour qu'un artefact puisse dire sa propre version sans qu'on interroge Git. Répéter une valeur, c'est accepter qu'elle diverge : le job **`version-consistency`** la contrôle dès la première étape (`scripts/ci/check_version.sh`). Restent volontairement à l'écart le `package.json` de la racine, qui décrit l'outillage et non l'application, et le champ `version` du chart, que la convention Helm distingue de l' `appVersion`.

Forme	Verdict	Raison
<code>v1.4.0</code> , <code>1.4.0</code> , <code>v1.4.0-alpha.1</code>	acceptée	SemVer valide
<code>v1.4</code> , <code>v1.4.0.1</code> , <code>latest</code>	refusée	ce n'est pas du SemVer
<code>v01.4.0</code>	refusée	SemVer interdit les zéros de tête : <code>01.4.0</code> et <code>1.4.0</code> seraient la même version sous deux tags
<code>v1.4.0+exp.sha.5114f85</code>	refusée	un tag Docker n'accepte pas le <code>+</code> ; le traduire en <code>_</code> casserait l'égalité entre tag Git et tag d'image
tag sur un commit jamais construit	refusée	le <code>docker pull</code> échoue

Deux conventions à retenir. Le **tag d'image ne porte pas le « v »** — le tag Git est `v1.4.0` , l'image est `back:1.4.0` , comme `node:22-alpine` . Et **`back:1.4.0` et `back:a1b2c3d` désignent le même digest**, puisque la promotion est un retag : déployer la version plutôt que le SHA ne change rien à ce qui tourne, ça change ce que le cluster affiche. Un `kubectl describe pod` en production nomme la version annoncée, sans table de correspondance à tenir.

État réel : un seul tag existe, `v1.0.0` . Le mécanisme est en place et testé (`run_tests.sh` , bloc `ci/promote_image.sh` , y compris l'assertion qui échoue si une promotion se met à construire) ; l'historique de versions, lui, tient en une ligne. **La génération automatique du changelog depuis les Conventional Commits n'est pas implémentée** (`RELEASE.md` §8, action A4.2 du plan).

7.4 Procédure de rollback

Deux niveaux, et ils ne répondent pas à la même situation.

Niveau	Outil	Déclencheur
Automatique	<code>scripts/deploy/deploy.sh</code>	Le rollout n'aboutit pas dans <code>\$DEPLOY_TIMEOUT</code> , ou échoue. Rien à faire, rien à décider
Manuel	<code>scripts/deploy/rollback.sh</code> , job <code>rollback-production</code>	Un bug est repéré après un déploiement qui avait pourtant réussi

Les étapes du rollback automatique, dans `deploy.sh` :

1. `kubectl set image` vers la version visée — devenu un no-op depuis que l'overlay éphémère pose déjà la bonne image à l' apply ; il est conservé pour l'attente et le garde-fou ;
2. `kubectl rollout status` , borné par `$DEPLOY_TIMEOUT` ;
3. si ça rate ou traîne, `kubectl rollout undo` — désactivable par `--no-auto-rollback` .

Le rollback manuel :

```
# Revenir à la version d'avant
bash scripts/deploy/rollback.sh -n "$PROD_NAMESPACE" -d back

# Revenir à une révision précise (liste : kubectl rollout history)
bash scripts/deploy/rollback.sh -n "$PROD_NAMESPACE" -d back --to-revision 7
```

Les vérifications.

- **Avant.** Le job commence par `exige_namespace` : un namespace de destination vide fait échouer le job au lieu de viser `default`.
- **Pendant.** `rollback.sh` attend lui aussi le rollout, et **prévient si le rollback lui-même échoue** — le cas le plus désagréable, et le seul qui doive réveiller quelqu'un.
- **Après.** `kubectl rollout history` indique vers quoi on est revenu. Comme chaque révision correspond à une image fixe taguée par SHA, on sait exactement ce qui tourne.
- **En continu.** Les deux mécanismes sont testés **à chaque commit** par `test-scripts`, avec un faux `kubectl` qui simule un déploiement raté : on vérifie que le `rollout undo` est bien déclenché, qu'il ne l'est **pas** quand tout va bien, et qu'on est prévenu si le rollback échoue.

Le défaut que ce dispositif a fallu corriger, parce qu'il est instructif. Tant que l'image passait par `kubectl set image` après un `apply` resté sur `PLACEHOLDER`, chaque déploiement insérait **deux** révisions — le placeholder puis la vraie image — et « la révision précédente » d'un déploiement sain était donc toujours le placeholder. `rollback-production` aurait **cassé la production au lieu de la réparer**. L'overlay éphémère supprime cette révision fantôme ; c'est ce qui rend le rollback fiable (K8S.md §6 et §14.10).

⚠ **Un réflexe à retenir, issu d'une observation sur cluster.** Enchaîner `rollback.sh` sans argument juste après un échec **redéploie l'image cassée** : le rollback automatique a déjà ramené la version saine, donc « la précédente » est redevenue la mauvaise.

Ce qui a réellement été exercé. Un `rollback-production` a été joué le 2026-09-23 à 13:34 et a réussi, suivi d'un redéploiement à 13:42 ; la production est en révision 4. Un premier rollback à 13:01 avait échoué sur un timeout du tunnel de l'agent. **En revanche, le rollback automatique de `deploy.sh` n'a jamais été déclenché par un vrai incident applicatif depuis la CI** : les échecs mesurés en CI portaient sur l'accès au cluster, pas sur l'application. Ce chemin n'est établi que par la campagne manuelle de K8S.md §14, sur une image volontairement cassée.

7.5 Les scripts d'automatisation

Toute la logique du pipeline vit dans `scripts/`, jamais dans des blocs YAML : un script est testable en local, relu par ShellCheck, et doublé de faux binaires qui permettent d'éprouver ses chemins d'échec.

Script	Langage	Objectif	Emplacement
<code>common.sh</code>	Bash	Fonctions communes : <code>log_info</code> / <code>warn/error</code> , <code>die</code> , <code>require_cmd</code> , <code>require_env</code> , <code>retry</code>	<code>scripts/lib/common.sh</code>
<code>build_and_push.sh</code>	Bash	Construit une image Docker et l'envoie au registry ; deux tags (SHA + tag mobile), scan Trivy optionnel	<code>scripts/ci/build_and_push.sh</code>
<code>promote_image.sh</code>	Bash	Pose le numéro SemVer sur une image déjà construite : pull, retag, push. Ne construit rien	<code>scripts/ci/promote_image.sh</code>
<code>check_version.sh</code>	Bash	Vérifie que les trois fichiers de version concordent avec le tag Git	<code>scripts/ci/check_version.sh</code>
<code>terraform_check.sh</code>	Bash	<code>fmt</code> , <code>validate</code> , <code>plan</code> et <code>apply</code> par environnement, avec <code>--require-plan</code>	<code>scripts/ci/terraform_check.sh</code>
<code>ansible_check.sh</code>	Bash	Contrôle les playbooks et les rôles (<code>ansible-lint</code>)	<code>scripts/ci/ansible_check.sh</code>
<code>deploy.sh</code>	Bash	Déploie sur Kubernetes, attend le rollout, revient en arrière tout seul si besoin	<code>scripts/deploy/deploy.sh</code>
<code>rollback.sh</code>	Bash	Revient à la révision précédente, ou à une révision nommée	<code>scripts/deploy/rollback.sh</code>
<code>quality_gate.py</code>	Python (stdlib)	Interroge l'API Sonar jusqu'au verdict, sort en 2 si la porte n'est pas franchie	<code>scripts/ci/quality_gate.py</code>
<code>check_coverage.py</code>	Python (stdlib)	Vérifie le taux de couverture du back contre <code>COVERAGE_MIN</code>	<code>scripts/ci/check_coverage.py</code>
<code>collect_dora.py</code>	Python (stdlib)	Calcule les quatre indicateurs DORA depuis l'API GitLab ; injecte dans Elasticsearch	<code>scripts/ci/collect_dora.py</code>
<code>notify.py</code>	Python (stdlib)	Annonce le résultat d'une étape sur un webhook d'équipe	<code>scripts/ci/notify.py</code>
<code>run_tests.sh</code>	Bash	266 assertions sur tous les scripts ci-dessus, sans cluster ni registry	<code>scripts/tests/run_tests.sh</code>
<code>validate_k8s.sh</code>	Bash	96 assertions sur les manifestes et le chart, sans cluster	<code>scripts/tests/validate_k8s.sh</code>
<code>run_k6.sh</code>	Bash	Lance les scénarios de performance en local, écrit un rapport JSON	<code>scripts/tests/run_k6.sh</code>

Deux partis pris expliquent ce tableau. Bash et Python stdlib, parce que les deux sont présents dans toutes les images CI, ne demandent aucune compilation, et restent lisibles par n'importe quel membre de l'équipe — et parce que se limiter à la bibliothèque standard évite tout `pip install` en CI, donc toute dépendance supplémentaire à auditer. Et `set -euo pipefail` **systématique** côté Bash, des codes de sortie distincts côté Python : un script de déploiement qui échoue mal est plus dangereux qu'un script absent.

7.6 Stratégie de backup

Il n'y a rien à sauvegarder, et ce n'est pas un oubli de procédure.

La base de MicroCRM vit dans la mémoire du processus. HSQldb est démarrée en mode `mem:` et alimentée à chaque démarrage par `InitialDataFixture`. Il n'existe ni fichier, ni volume, ni instantané : un pod qui redémarre repart d'une base vide, aussitôt regarnie des mêmes données de démonstration.

Autrement dit, **une procédure de sauvegarde n'aurait rien à copier.** Écrire un `CronJob` de `pg_dump` sur cette application ne serait pas une sécurité, ce serait **un décor** — et c'est le genre de décor qu'un jury repère.

Élément	Fréquence	Rétention	Justification
Données applicatives	sans objet	sans objet	La base vit en mémoire ; il n'existe aucun fichier à copier
Code, manifestes, IaC, dashboards	à chaque commit	historique Git complet	Tout vit dans le dépôt, y compris les tableaux de bord Kibana exportés en NDJSON
Images applicatives	à chaque pipeline	registry GitLab, tag = SHA	Immuables, donc restaurer une version = redéployer un tag existant
État Terraform	à chaque <code>apply</code>	versions conservées par GitLab	Backend <code>http</code> , un état par environnement, verrouillé
Données Elasticsearch (logs)	aucune sauvegarde	aucune rétention	PVC de 5 Gio sur le disque du poste, pas d'ILM. Les seuils disque (85/90/95 %) mettent l'index en lecture seule bien avant que le volume soit plein

La vraie garantie de ce projet n'est pas une sauvegarde de données : c'est que l'environnement se reconstruit intégralement depuis le seul dépôt. Et c'est vérifiable — la procédure a été **exécutée de bout en bout le 2026-09-22**, à partir d'une destruction réelle (`RELEASE.md` §9.4 et §9.5) :

```
kubectl delete namespace "$NAMESPACE"           # 0. destruction réelle
cd ansible && ansible-playbook site.yml             # 1. poste et cluster
cd terraform/environments/staging && terraform apply # 2. namespace, quota, policies
kubectl apply -k k8s/overlays/staging -n "$NAMESPACE" # 3. application
curl -s http://127.0.0.1:18081/persons           # 4. vérification
```

Résultat consigné : `ansible ok=23 changed=0 failed=0` , **6 ressources Terraform créées**, rollout des deux Deployments en **11 s**, `/persons` qui répond et `/actuator/health` en `{"status":"UP"}` .

Objectif	Valeur	Pourquoi
RTO	Non formalisé	Aucune cible de temps de rétablissement n'est écrite nulle part dans le dépôt. La seule mesure approchante est le <code>time_to_restore_service</code> DORA : médiane 2,14 h sur 2 observations , ce qui est un constat, pas un engagement
RPO	Sans objet	Une perte de données maximale tolérée suppose des données à perdre. Il n'y en a pas : la base est jetable et recrée à l'identique à chaque démarrage

Ce qu'il faudrait pour qu'il y ait quelque chose à sauvegarder, chiffré plutôt que laissé en intention (`RELEASE.md` §9.2) : remplacer HSQLDB par PostgreSQL (S), déployer la base en `StatefulSet` + PVC (M), sortir les identifiants dans un `Secret` (S), **introduire Flyway ou Liquibase** (M), lever le plafond de replicas (S), recalculer les quotas (S). Le point le moins évident est le quatrième : tant que la base est jetable, `hibernate.ddl-auto` peut la recréer à chaque démarrage ; **dès qu'elle persiste, cette commodité devient un danger**.

Et trois règles vaudraient dès le premier jour : la sauvegarde part **hors du cluster** (un instantané qui vit sur le volume qu'il sauvegarde ne sauvegarde rien), elle est **chiffrée** puisqu'elle contient des données personnelles, et **elle est restaurée périodiquement**. Une sauvegarde jamais restaurée n'est pas une sauvegarde, c'est une croyance.

7.7 Monitoring et alerting

Il faut distinguer deux choses que le mot « alerting » confond : la surveillance de la chaîne, et la surveillance de l'application.

Ce qui existe : la supervision par les logs

Brique	Rôle
Filebeat	DaemonSet, provider <code>autodiscover kubernetes</code> . Filtre deux fois sur le namespace observé : ce cluster héberge aussi des namespaces tiers
Elasticsearch	Data stream <code>microcrm-logs-AAAA.MM.JJ</code> , PVC de 5 Gio
Kibana	6 panneaux — volume, latence, erreurs applicatives et HTTP, statuts, logs récents — exportés en NDJSON et versionnés dans <code>k8s/elk/dashboards/</code>
<code>collect_dora.py</code>	Les 4 indicateurs DORA, calculés depuis l'API GitLab, injectables dans l'index <code>microcrm-dora</code>
Sondes K8s	<code>startupProbe</code> , <code>livenessProbe</code> , <code>readinessProbe</code> sur les deux Deployments, via Actuator

Les métriques et seuils réellement disponibles :

Métrique	Source	Valeur observée	Seuil
Latence du front (p50/p95/p99)	Journal d'accès de Caddy	0,14 ms / 1,60 ms / 3,11 ms (105 requêtes)	aucun seuil d'alerte
Volume de logs par conteneur	Kibana	—	aucun
Erreurs applicatives	<code>log.level</code> du back (ECS)	—	aucun
p95 des lectures / écritures	k6, en CI	—	500 ms / 800 ms — bloquants en pipeline, pas en exploitation
DORA (4 indicateurs)	<code>collect_dora.py</code>	0,1667/j · 1,38 h · 2,14 h · 66,67 %	cibles au §2.2, non automatisées

⚠ **La latence mesurée est celle du serveur web, pas de l'API.** Caddy sert le bundle Angular et `/config.json` ; les appels à l'API partent du navigateur vers un hôte distinct et ne passent pas par lui. Mesurer la latence de l'API demanderait de l'instrumenter — c'est le domaine des métriques, et ces métriques n'existent pas.

Ce qui n'existe pas

Volet	État	Raison
Métriques système (CPU, mémoire, latence d'API)	Non implémenté	Ni Prometheus, ni <code>metrics-server</code> , ni <code>micrometer-registry-prometheus</code> . Le projet collecte des logs, pas des métriques
Alerting applicatif	Non implémenté	Rien ne prévient : la supervision se consulte, elle ne réveille personne (<code>QUALITY.md</code> §6)
Seuils d'alerte en exploitation	Non définis	Conséquence directe de la ligne précédente : il n'y a pas de règle à seuiller
Montée en charge automatique	Non implémenté	Sans métriques, il manque jusqu'au signal sur lequel un autoscaler déciderait
Rétention des logs (ILM)	Non implémenté	Les données s'accumulent jusqu'aux seuils disque d'Elasticsearch
Sécurité de la stack ELK	Désactivée	<code>xpack.security.enabled: false</code> , aucun Ingress sur Kibana, accès par <code>port-forward</code> . Assumé pour une stack locale, inacceptable ailleurs
Collecte DORA automatisée	Non implémenté	Aucun job de CI n'exécute <code>collect_dora.py</code> — vérifié, aucun fichier de <code>.gitlab/ci/</code> ne l'appelle. Il se lance à la main (action A3.3 du plan)

Ce qui existe depuis peu : la notification de pipeline

C'est le seul mécanisme d'alerte du projet, et il porte sur la **chaîne**, pas sur l'application.

Mécanisme	Quand il se déclenche	Vers où
Job <code>notify-echec</code>	<code>when: on_failure</code> , dans la dernière étape (<code>infra-apply</code>), sur <code>develop</code> , <code>main</code> et les tags	Webhook <code>\$NOTIFY_WEBHOOK_URL</code> (Slack, Mattermost, Discord...)
<code>after_script</code> de <code>.deploy_template</code>	Chaque déploiement, succès comme échec	Même webhook

Trois partis pris, et chacun a sa raison (`scripts/ci/notify.py`) :

1. **On ne notifie que les échecs — sauf les déploiements.** Un canal qui annonce chaque pipeline vert devient un canal qu'on n'ouvre plus, et le jour où il annonce un échec, personne ne le lit. Les déploiements font exception parce qu'une mise en production réussie est une information que l'équipe attend, pas du bruit.
2. **Webhook absent = rien à faire, et surtout pas un échec.** Le script sort en 0 avec une trace dans le journal. Sans cela, un dépôt cloné sans la variable verrait tous ses pipelines rougir sur une notification non configurée — et on apprendrait à ignorer les jobs rouges.
3. **Une notification qui échoue ne fait jamais échouer le pipeline.** Un canal indisponible ne dit rien sur la qualité du déploiement. `allow_failure: true` est la ceinture, le code de sortie 0 les bretelles.

`notify-echec` est placé dans la dernière étape délibérément : un `when: on_failure` y attrape l'échec de **n'importe laquelle** des étapes précédentes. Placé plus tôt, il ne verrait que ce qui le précède, et un déploiement raté passerait sous silence.

En résumé, la distinction à ne pas perdre. Un pipeline qui casse prévient quelqu'un depuis aujourd'hui. **Une application qui tombe en production ne prévient toujours personne.** Les deux sont des sujets différents, et seul le premier est traité.

8. Annexes

8.1 Les documents détaillés du dépôt

Document	Ce qu'on y trouve
AUDIT.md	Audit du processus initial, SWOT, frictions, plan de sécurité (§7), traitement des vulnérabilités (§7.4), politique de dépendances (§7.5)
VEILLE.md	Veille technologique : chaque outil comparé, justifié, chiffré
schema.md	Chaîne cible, table de normalisation cycle → GitLab CI, et ce que la chaîne ne fait pas encore
QUALITY.md	Plan de tests et de sécurité : Sonar, SpotBugs, Dependency-Check, Trivy, k6, couverture et mutation
ARCHITECTURE.md	Architecture applicative et plateforme, schémas laC, option locale (§9) et transposition cloud (§10-11)
RELEASE.md	Versionnage SemVer, déploiement, rollback, sauvegarde et restauration (§9)
K8S.md	Manifestes, overlays, sondes, sécurité des conteneurs, accès au registry (§12), campagnes de déploiement réel (§14)
HELM.md	Le chart, et pourquoi il vient en plus de Kustomize
TERRAFORM.md	Modules, état partagé, frontière de responsabilité, limites assumées
ANSIBLE.md	Provisionnement du poste et du cluster
MONITORING.md	Stack ELK, tableaux de bord, indicateurs DORA (§9)
SCRIPTS.md	Chaque script : but, fonctionnement, paramètres, codes de sortie
VARIABILISATION.md	Ce qui est externalisé, et pourquoi
GUIDE.md	Prise en main rapide du dépôt
docs/pipeline-ci.md	Le découpage du pipeline et les pièges de <code>include:</code> / <code>extends:</code>
docs/plan-optimisation-release.md	Plan d'optimisation par vagues, avec porteur, effort et preuve d'atteinte
docs/documentation-infrastructure.md	Livrable d'infrastructure (PDF associé)
docs/rapport-performance.md	Livrable de performance : DORA, tests, sécurité, supervision, gains (PDF associé)
docs/schema-architecture.md	Les huit schémas de l' architecture cible — à ne pas confondre avec l'état implémenté

Sources des schémas. `docs/schemas/*.mmd` , un fichier par schéma. Les rendre isolément :

```
docker run --rm -v "$PWD:/data" minlag/mermaid-cli \
-i /data/docs/schemas/<nom>.mmd -o /data/<nom>.svg
```

8.2 Glossaire

Terme	Définition dans ce projet
Clean as You Code	Principe Sonar : la Quality Gate ne s'applique qu'au code nouveau ou modifié, jamais à l'existant
DORA (4 indicateurs)	Fréquence de déploiement, délai de mise en production, temps de rétablissement, taux d'échec des changements
Mutant / score de mutation	PIT modifie le bytecode ; si aucun test ne devient rouge, le mutant « survit » et la ligne est couverte sans être vérifiée
Overlay éphémère	<code>kustomization.yaml</code> écrit à la volée par le job de déploiement, qui ajoute le chemin de registry et le tag d'image aux manifestes du dépôt
Promotion	Poser un tag SemVer sur une image déjà construite et scannée, par retag — jamais par reconstruction
Quality Gate	Verdict binaire de Sonar sur le code neuf, récupéré par <code>quality_gate.py</code> qui sort en 2 s'il est rouge
SCA	<i>Software Composition Analysis</i> — analyse des dépendances tierces (ici OWASP Dependency-Check)
SAST	<i>Static Application Security Testing</i> — analyse statique orientée sécurité (ici Sonar + Find-Sec-Bugs)
Shift-left	Placer un contrôle au plus tôt : ici <code>security</code> s'exécute avant <code>build</code>
Stub / faux binaire	Faux <code>kubectl</code> , <code>docker</code> ou <code>trivy</code> placés en tête de <code>PATH</code> par la suite de tests, pour éprouver les chemins d'échec sans cluster

8.3 Reproduire les vérifications de ce document

```
# Les scripts du pipeline – 266 assertions, sans cluster ni registry
bash scripts/tests/run_tests.sh

# Les manifestes et le chart – 96 assertions, sans cluster
bash scripts/tests/validate_k8s.sh

# La couverture et la mutation du back
cd back && ./gradlew test jacocoTestReport pitest

# Les indicateurs DORA, contre l'API GitLab (projet miroir public, sans jeton)
python3 scripts/ci/collect_dora.py --project 84606666 --days 30

# L'infrastructure, hors cluster
cd terraform/environments/staging && terraform init -backend=false && terraform validate
```

8.4 Écarts connus entre documents du dépôt

Cette section existe parce qu'un document de synthèse qui masque les désaccords de ses sources ne vaut rien. Chacun a été vérifié dans le dépôt.

Écart	Arbitrage retenu ici
L'état des déploiements : plusieurs documents de la copie de travail portent « aucun déploiement réussi depuis la CI », « 7 tentatives, 7 échecs », « quota épuisé »	Périmé. <code>develop</code> a été actualisée le 2026-09-23 (commit <code>c17249c</code> , PR #26) : 5 déploiements réussis, rollback joué, runner auto-hébergé. Les captures de <code>docs/captures/</code> datées du 2026-09-23 le confirment
Nombre d'étapes du pipeline : <code>QUALITY.md</code> en tête annonce 8 stages	10 stages , vérifié dans <code>.gitlab-ci.yml</code> (<code>infra</code> et <code>infra-apply</code> manquent à cette liste)
Nombre d'assertions : <code>ARCHITECTURE.md</code> , <code>GUIDE.md</code> , <code>K8S.md</code> et <code>docs/rapport-performance.md</code> annoncent 151	266 , obtenu en exécutant <code>run_tests.sh</code> le 2026-09-24. Le chiffre 151 est antérieur aux blocs de tests ajoutés depuis
Nombre de jobs dans <code>docs/schemas/plateforme-deploiement.mmd</code> : 30 en copie de travail, 36 sur <code>develop</code>	37 , compté job par job dans les 13 fichiers de <code>.gitlab/ci/</code> — <code>notify-echec</code> a été ajouté depuis
Seuil de mutation : <code>QUALITY.md</code> donne <code>MUTATION_MIN=90</code> « comme en CI » dans un exemple, et 80 % dans son tableau	80 , valeur réelle de <code>.gitlab/ci/variables.yml</code> et défaut de <code>back/build.gradle</code>
Taille de l'image back : 399 Mo au §3 d' <code>ARCHITECTURE.md</code> , 377 Mo dans ses limites connues	Non tranché — les deux chiffres coexistent dans le même document, probablement pris à deux moments différents
<code>TERRAFORM.md</code> §9.4 : « l' <code>apply</code> n'a pas été joué » ; §9.5 : « les jobs de déploiement utilisent encore <code>\$KUBE_CONFIG</code> »	Périmé. <code>RELEASE.md</code> §9.5 consigne un <code>apply</code> réel le 2026-09-22 (6 ressources créées) et le passage des jobs de déploiement par l'agent est acté au 2026-09-23
<code>docs/rapport-performance.md</code> §8 : « la restauration complète d'un environnement est documentée, jamais jouée »	Contredit par <code>RELEASE.md</code> §9.4-9.5, qui en donne le compte rendu daté du 2026-09-22
Namespace de production : <code>microcrm-prod</code> dans le schéma d' <code>ARCHITECTURE.md</code> §8.1, <code>microcrm-production</code> dans <code>terraform/environments/production/terraform.tfvars</code>	<code>microcrm-production</code> , la valeur réellement appliquée
<code>VEILLE.md</code> §8 range l'laC dans les « pistes non retenues à ce stade »	Périmé : Terraform et Ansible sont implémentés. <code>VEILLE.md</code> décrit l'état de la veille au moment où elle a été faite
<code>docs/schema-architecture.md</code> décrit PostgreSQL, un staging automatique et des images signées	Ce document décrit la cible, pas l'implémenté. L'état réel est HSQldb en mémoire, staging manuel, images non signées

8.5 Les limites que ce document n'a pas contournées

Récapitulatif de tout ce qui porte « non implémenté » ou « non mesuré » dans les pages précédentes, avec le renvoi vers sa justification.

Sujet	État	Où c'est justifié
Tests E2E (Cypress, Playwright)	Non implémenté	§4.1
Environnement de développement déployé	Non implémenté	§5.3
Fournisseur cloud, VPC, subnets, services managés	Volontairement absent	§5.1
<code>npm audit</code> / SCA côté front	Non implémenté	§6.2
Signature d'images et attestation de provenance	Non implémenté	§6.2
Politique de rotation des secrets	Non formalisée	§6.3
Délais de traitement des CVE MEDIUM et LOW	Non définis	§6.4
Mise à jour automatisée des dépendances (Renovate)	Non implémenté	§6.4
Déploiement progressif (blue/green, canary)	Non implémenté	§7.1
Génération automatique du changelog	Non implémenté	§7.3
Déploiement staging automatique (<code>on_success</code>)	Non implémenté	§5.3, §7.2
RTO	Non formalisé	§7.6
RPO	Sans objet	§7.6
Sauvegarde de données applicatives	Sans objet	§7.6
Métriques système (Prometheus, Grafana)	Non implémenté	§7.7
Alerting applicatif	Non implémenté	§7.7
Rétention des logs (ILM)	Non implémenté	§7.7
Collecte DORA exécutée par la CI	Non implémenté	§7.7
Approbations et protection de branche	Non formalisées	§3.2
Branches <code>release/*</code>	Jamais utilisées	§3.2
<code>NetworkPolicy</code> effectives	Décrites, non prouvées (CNI minikube)	§5.1
Chart Helm déployé	Rendu et comparé, jamais appliqué	§5.4